# Task

Analyze the "inj.csv" dataset to understand football injury patterns, build and optimize an SVM classification model to predict injury outcomes, and summarize key findings, including factors contributing to injuries.

# Load Data and Initial Exploration

## Subtask:

Load the 'inj.csv' dataset into a pandas DataFrame. Display the first few rows, column information, and basic descriptive statistics to understand its structure and initial characteristics. This is the start of the EDA phase.

**Reasoning**: To load the dataset and perform an initial exploration, I will use pandas to read the CSV file, then display the first few rows, column information, and descriptive statistics.

```
import pandas as pd

df = pd.read_csv('/content/inj.csv')

print('First 5 rows of the DataFrame:')
print(df.head())

print('\nDataFrame Information:')
df.info()

print('\nDescriptive Statistics:')
print(df.describe())
```

```
First 5 rows of the DataFrame:
  Season                     Injury     Days  Games missed injury_from_parsed  \
0  20/21  Syndesmosis ligament tear  43 days             9          1/28/2021
1  20/21                Knee injury  37 days             6           3/7/2021
2  20/21               Corona virus  21 days             4          2/17/2021
3  20/21                      bruise   8 days             2          11/6/2020
4  20/21            Ligament injury  22 days             2          7/26/2020

  injury_until_parsed           player_name  player_age player_position  \
0           3/11/2021       Alexander Nübel          24      Goalkeeper
1           4/12/2021  Ron-Thorben Hoffmann          22      Goalkeeper
2            3/9/2021       Benjamin Pavard          25     Centre-Back
3          11/13/2020       Benjamin Pavard          25     Centre-Back
4           8/16/2020       Benjamin Pavard          25     Centre-Back

            club      league
0  Bayern Munich  Bundesliga
1  Bayern Munich  Bundesliga
2  Bayern Munich  Bundesliga
3  Bayern Munich  Bundesliga
4  Bayern Munich  Bundesliga

DataFrame Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15603 entries, 0 to 15602
Data columns (total 11 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Season               15603 non-null  object
 1   Injury               15603 non-null  object
 2   Days                 15603 non-null  object
 3   Games missed         15603 non-null  int64
 4   injury_from_parsed   15603 non-null  object
 5   injury_until_parsed  15603 non-null  object
 6   player_name          15603 non-null  object
 7   player_age           15603 non-null  int64
 8   player_position      15603 non-null  object
 9   club                 15603 non-null  object
 10  league               15603 non-null  object
dtypes: int64(2), object(9)
memory usage: 1.3+ MB

Descriptive Statistics:
       Games missed    player_age
```

```
count  15603.000000  15603.000000
mean       5.507146     26.550663
std        7.635735      4.398164
min        1.000000     16.000000
25%        1.000000     23.000000
50%        3.000000     26.000000
75%        6.000000     30.000000
max      145.000000     43.000000
```

## Missing Values and Data Cleaning

### Subtask:

Identify and handle any missing values in the dataset. This might involve imputation, dropping rows/columns, or other appropriate cleaning strategies based on the nature of the missing data.

To ensure the dataset is ready for analysis, I will address potential missing values and convert columns to appropriate data types. Specifically, the 'Days' column needs to be converted from a string format (e.g., '43 days') to a numerical integer to allow for quantitative analysis of injury duration. The 'injury_from_parsed' and 'injury_until_parsed' columns, which are currently objects, will be converted to datetime objects to enable time-series analysis and calculations involving dates.

**Reasoning**: Now, I will implement the data cleaning steps as outlined in the instructions: converting the 'Days' column to an integer and the 'injury_from_parsed' and 'injury_until_parsed' columns to datetime objects. I will then display the `info()` and `head()` of the dataframe to confirm the changes.

```
df['Days'] = df['Days'].str.replace(' days', '').astype(int)
df['injury_from_parsed'] = pd.to_datetime(df['injury_from_parsed'])
df['injury_until_parsed'] = pd.to_datetime(df['injury_until_parsed'])

print('DataFrame Information after type conversion:')
df.info()

print('\nFirst 5 rows of the DataFrame after type conversion:')
print(df.head())
```

```
DataFrame Information after type conversion:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15603 entries, 0 to 15602
Data columns (total 11 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Season               15603 non-null  object
 1   Injury               15603 non-null  object
 2   Days                 15603 non-null  int64
 3   Games missed         15603 non-null  int64
 4   injury_from_parsed   15603 non-null  datetime64[ns]
 5   injury_until_parsed  15603 non-null  datetime64[ns]
 6   player_name          15603 non-null  object
 7   player_age           15603 non-null  int64
 8   player_position      15603 non-null  object
 9   club                 15603 non-null  object
 10  league               15603 non-null  object
dtypes: datetime64[ns](2), int64(3), object(6)
memory usage: 1.3+ MB

First 5 rows of the DataFrame after type conversion:
  Season                     Injury  Days  Games missed injury_from_parsed  \
0  20/21  Syndesmosis ligament tear    43             9         2021-01-28
1  20/21                Knee injury    37             6         2021-03-07
2  20/21               Corona virus    21             4         2021-02-17
3  20/21                     bruise     8             2         2020-11-06
4  20/21            Ligament injury    22             2         2020-07-26

  injury_until_parsed           player_name  player_age player_position  \
0          2021-03-11       Alexander Nübel          24      Goalkeeper
1          2021-04-12  Ron-Thorben Hoffmann          22      Goalkeeper
2          2021-03-09       Benjamin Pavard          25     Centre-Back
3          2020-11-13       Benjamin Pavard          25     Centre-Back
4          2020-08-16       Benjamin Pavard          25     Centre-Back

            club      league
0  Bayern Munich  Bundesliga
1  Bayern Munich  Bundesliga
2  Bayern Munich  Bundesliga
3  Bayern Munich  Bundesliga
```

```
4  Bayern Munich  Bundesliga
```

## Exploratory Data Analysis (EDA)

### Subtask:

Conduct further exploratory data analysis. This includes visualizing key distributions of relevant features (e.g., injury types, player positions, body parts affected, if available in the data). Use visualizations to uncover patterns and relationships within the data.

**Reasoning**: I need to import the `matplotlib.pyplot` and `seaborn` libraries to perform the requested visualizations for the Exploratory Data Analysis (EDA).

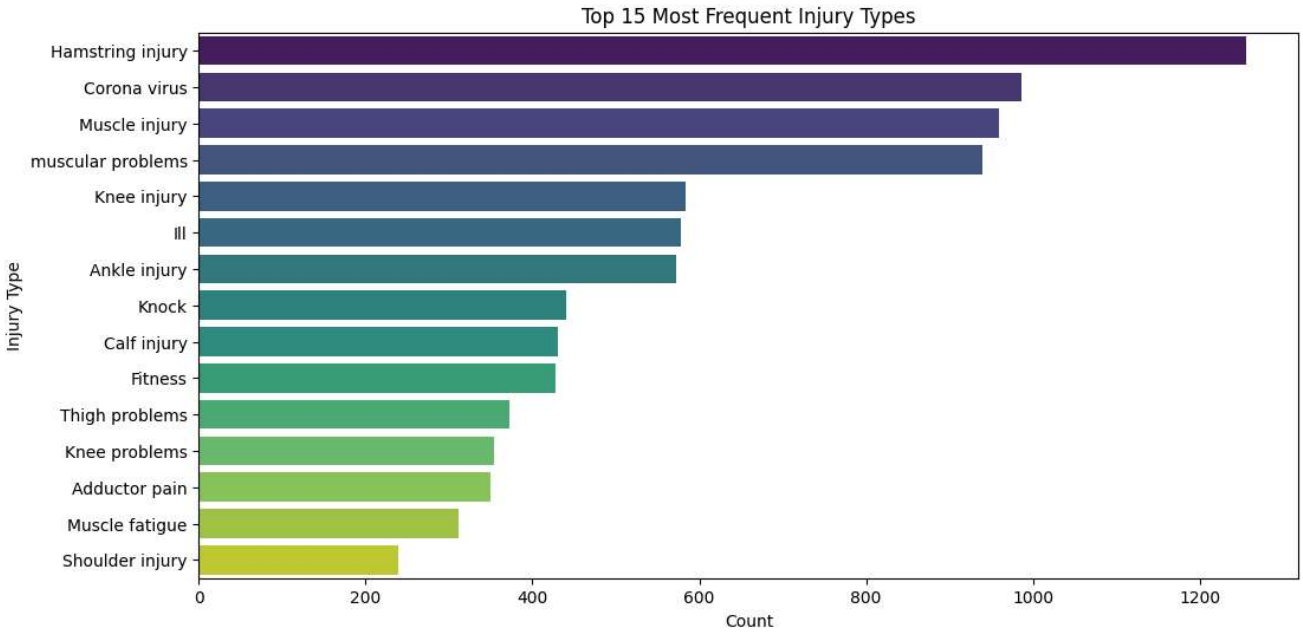
```
import matplotlib.pyplot as plt
import seaborn as sns

print("Libraries 'matplotlib.pyplot' and 'seaborn' imported successfully.")
```

```
Libraries 'matplotlib.pyplot' and 'seaborn' imported successfully.
```

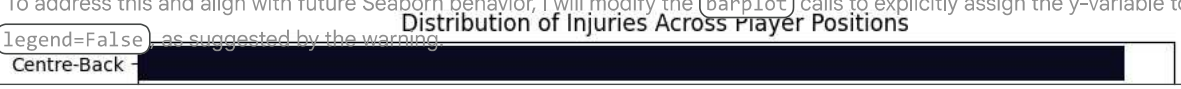
**Reasoning**: I will create visualizations as per the subtask instructions: bar plots for injury types and player positions, and histograms and box plots for 'Days' and 'Games missed' to explore their distributions and identify patterns.

```
plt.figure(figsize=(12, 6))
injury_counts = df['Injury'].value_counts().head(15)
sns.barplot(y=injury_counts.index, x=injury_counts.values, palette='viridis')
plt.title('Top 15 Most Frequent Injury Types')
plt.xlabel('Count')
plt.ylabel('Injury Type')
plt.show()

plt.figure(figsize=(10, 6))
position_counts = df['player_position'].value_counts()
sns.barplot(y=position_counts.index, x=position_counts.values, palette='magma')
plt.title('Distribution of Injuries Across Player Positions')
plt.xlabel('Count')
plt.ylabel('Player Position')
plt.show()

plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
sns.histplot(df['Days'], bins=30, kde=True, color='skyblue')
plt.title('Distribution of Injury Duration (Days)')
plt.xlabel('Days')
plt.ylabel('Frequency')

plt.subplot(1, 2, 2)
sns.boxplot(y=df['Days'], color='skyblue')
plt.title('Box Plot of Injury Duration (Days)')
plt.ylabel('Days')
plt.tight_layout()
plt.show()

plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
sns.histplot(df['Games missed'], bins=30, kde=True, color='lightcoral')
plt.title('Distribution of Games Missed')
plt.xlabel('Games Missed')
plt.ylabel('Frequency')

plt.subplot(1, 2, 2)
sns.boxplot(y=df['Games missed'], color='lightcoral')
plt.title('Box Plot of Games Missed')
plt.ylabel('Games Missed')
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_1067/1769180013.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set

  sns.barplot(y=injury_counts.index, x=injury_counts.values, palette='viridis')
```



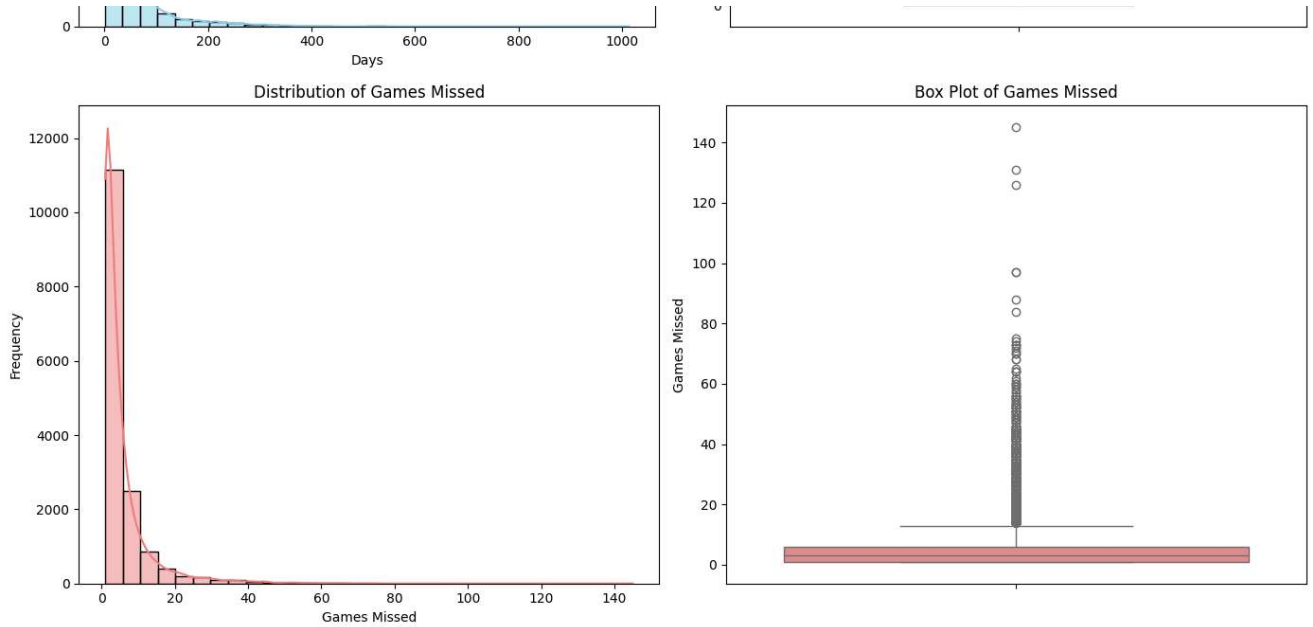

```
/tmp/ipykernel_1067/1769180013.py:11: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set

  sns.barplot(y=position_counts.index, x=position_counts.values, palette='magma')
```

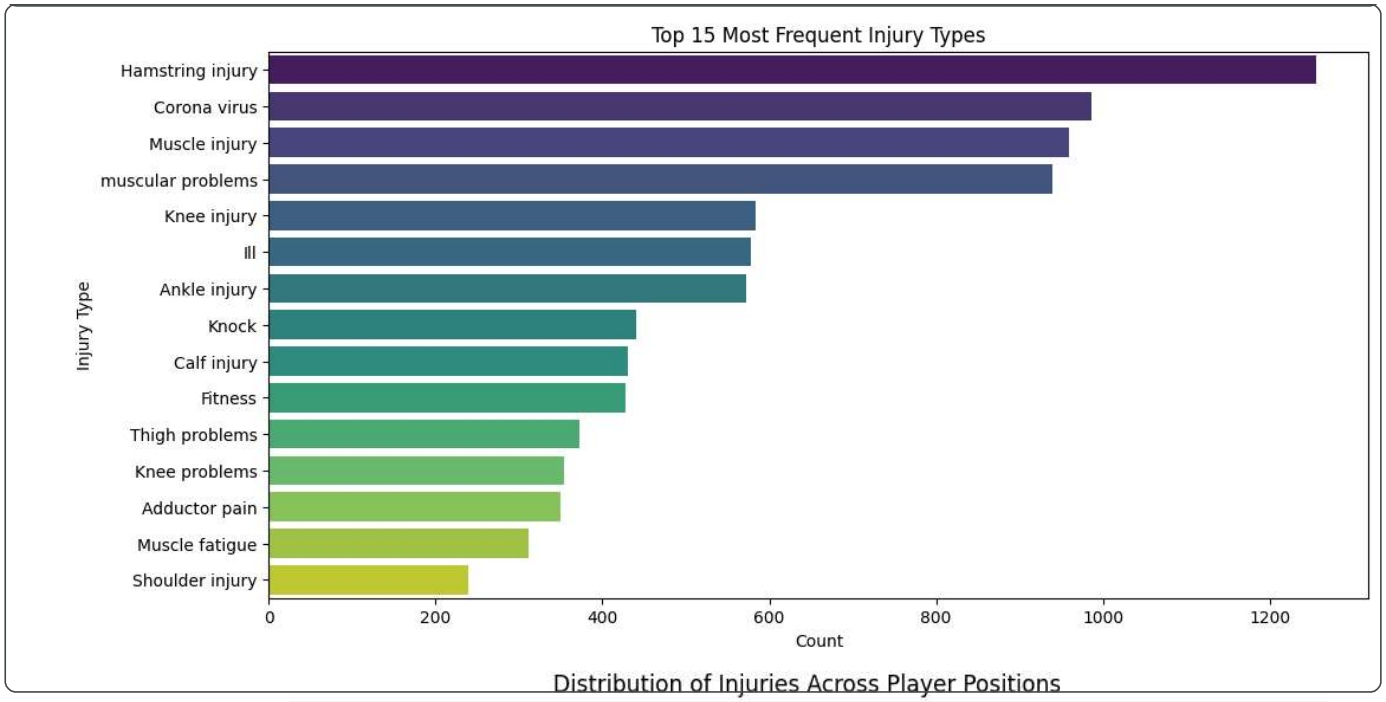


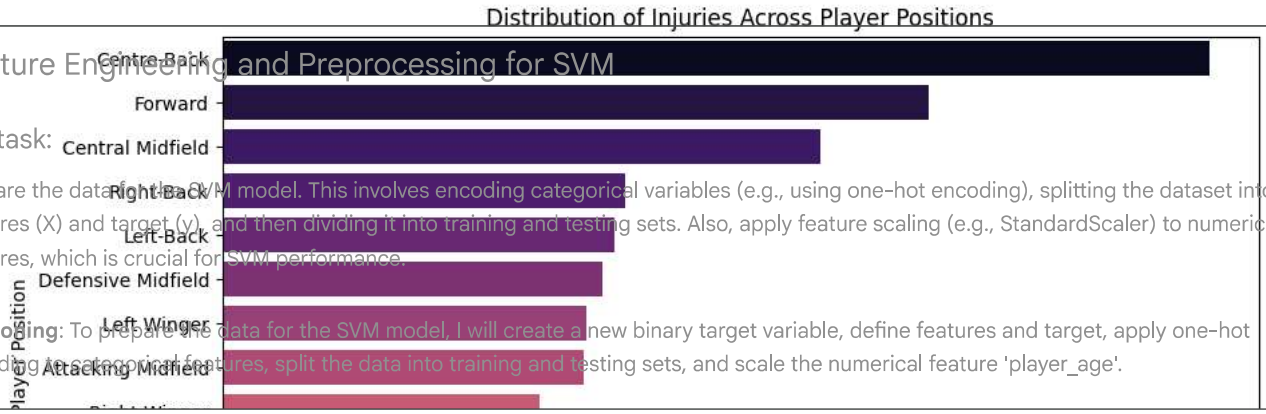

**Reasoning**: The previous code generated `FutureWarning` messages for `sns.barplot` when `palette` was used without assigning a `hue` variable. To address this and align with future Seaborn behavior, I will modify the `barplot` calls to explicitly assign the y-variable to `hue` and set `legend=False`, as suggested by the warning.

```
plt.figure(figsize=(12, 6))
injury_counts = df['Injury'].value_counts().head(15)
sns.barplot(y=injury_counts.index, x=injury_counts.values, hue=injury_counts.index, palette='viridis', legend=False)
plt.title('Top 15 Most Frequent Injury Types')
plt.xlabel('Count')
plt.ylabel('Injury Type')
plt.show()

plt.figure(figsize=(10, 6))
position_counts = df['player_position'].value_counts()
sns.barplot(y=position_counts.index, x=position_counts.values, hue=position_counts.index, palette='magma', legend=False)
plt.title('Distribution of Injuries Across Player Positions')
plt.xlabel('Count')
plt.ylabel('Player Position')
plt.show()

plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
sns.histplot(df['Days'], bins=30, kde=True, color='skyblue')
plt.title('Distribution of Injury Duration (Days)')
plt.xlabel('Days')
plt.ylabel('Frequency')

plt.subplot(1, 2, 2)
sns.boxplot(y=df['Days'], color='skyblue')
plt.title('Box Plot of Injury Duration (Days)')
plt.ylabel('Days')
plt.tight_layout()
plt.show()

plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
sns.histplot(df['Games missed'], bins=30, kde=True, color='lightcoral')
plt.title('Distribution of Games Missed')
plt.xlabel('Games Missed')
plt.ylabel('Frequency')

plt.subplot(1, 2, 2)
```

```
sns.boxplot(y=df['Games missed'], color='lightcoral')
plt.title('Box Plot of Games Missed')
plt.ylabel('Games Missed')
plt.tight_layout()
plt.show()
```



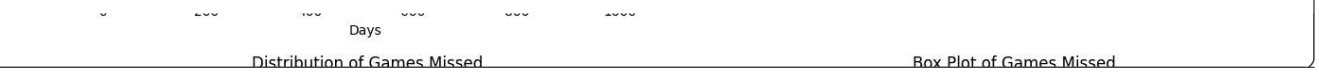



## Feature Engineering and Preprocessing for SVM

### Subtask:

Prepare the data for the SVM model. This involves encoding categorical variables (e.g., using one-hot encoding), splitting the dataset into features (X) and target (y), and then dividing it into training and testing sets. Also, apply feature scaling (e.g., StandardScaler) to numerical features, which is crucial for SVM performance.

**Reasoning**: To prepare the data for the SVM model, I will create a new binary target variable, define features and target, apply one-hot encoding to categorical features, split the data into training and testing sets, and scale the numerical feature 'player_age'.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Create a new binary target variable 'long_term_injury'
df['long_term_injury'] = (df['Days'] > 30).astype(int)

# 2. Define the feature matrix X and target vector y
X = df[['Season', 'Injury', 'player_age', 'player_position', 'league']]
y = df['long_term_injury']

# 3. Identify categorical columns
categorical_cols = ['Season', 'Injury', 'player_position', 'league']

# 4. Apply one-hot encoding to categorical columns
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# 5. Split the preprocessed X and y into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 6. Initialize StandardScaler
scaler = StandardScaler()

# 7. Apply StandardScaler to the numerical feature 'player_age'
# Note: 'player_age' is a single column, so we need to pass it as a DataFrame slice
X_train['player_age'] = scaler.fit_transform(X_train[['player_age']])
X_test['player_age'] = scaler.transform(X_test[['player_age']])

# 8. Print the shapes of the resulting datasets
print('Shape of X_train:', X_train.shape)
print('Shape of X_test:', X_test.shape)
print('Shape of y_train:', y_train.shape)
print('Shape of y_test:', y_test.shape)
```